Concepte fundamentale ale limbajelor de programare

Object-Oriented Programming Languages
Curs 11

conf. dr. ing. Ciprian-Bogdan Chirila

Universitatea Politehnica Timisoara
Departamentul de Calculatoare si Tehnologia Informatiei

4 decembrie 2024

FCPL - 11 - Object-Oriented Programming Languages 2

1. Object-oriented programming in C#
   - Introduction
   - Example
2. Object-oriented programming in Python
   - Python Classes and Objects
   - OOP Principles
   - Advanced OOP Features
3. Object-oriented programming in JavaScript
   - Objects in JavaScript
   - Classes and Objects
   - Encapsulation
   - Inheritance

Object-oriented programming in C#

- Built by Microsoft for .NET platform
- Like Java is derived from C and C++
- Portability concept taken from Java
  - Based on Microsoft intermediate language – MSIL
  - .NET framework as virtual machine
- Programming in multiple PLs
- Each part of the program can be written in the most expressive PL
- Full integration with the Windows platform

Object-oriented programming in C#

- C# programs start in the Main function
- C# system contain classes
- Organized in hierarchies by inheritance
- Regarding inheritance
  - Superclass or base class
  - Subclass or derived class
- Multiple inheritance is not allowed
  - A class may not have multiple superclasses

Example in C#

```
using System;

// A class representing bi-dimensional objects.
class Shape2D
{
 public double width;
 public double height;
 public void showDim()
 {
  Console.WriteLine("Width and height are " +
  width +" and " + height);
 }
}
```

Example in C#

```
// class Triangle derives from class Shape2D.
class Triangle : Shape2D
{
 public string typeOfTriangle;
 public double area()
 {
  return width * height / 2;
 }

 public void showType ()
 {
  Console.WriteLine("Triangle is " + typeOfTriangle);
 }
}
```

Example in C#

```
class Shapes
{
 public static void Main()
 {
  Triangle t1 = new Triangle();
  t1.width = 4.0;
  t1.height = 4.0;
  t1.typeOfTriangle = "isosceles";

  Triangle t2 = new Triangle();
  t2.width = 8.0;
  t2.height = 12.0;
  t2.typeOfTriangle = "rectangular";
```

Example in C#

```
  ConsoleWriteln("Information about t1: ");
  t1.showType();
  t1.showDim() ;

  ConsoleWriteLine("Triangle area is " + t1.aria());
  ConsoleWriteLine();

  ConsoleWriteln("Information about t2: ");
  t2.showType();
  t2.showDim() ;
  ConsoleWriteLine("Triangle area is " + t2.aria());
 }
}
```

Object-oriented programming in C#

- Class Shape2D
  - Defines the generic shape attributes
  - Square
  - Rectangle
  - Triangle
- Class Triangle
  - Is derived from Shape2D
  - One attribute added: typeOfTriangle
  - two methods added: area(), showType()
  - Is able to refer attributes of Shape2D as its own

FCPL - 11 - Object-Oriented Programming Languages 2

1. Object-oriented programming in C#
   - Introduction
   - Example
2. Object-oriented programming in Python
   - Python Classes and Objects
   - OOP Principles
   - Advanced OOP Features
3. Object-oriented programming in JavaScript
   - Objects in JavaScript
   - Classes and Objects
   - Encapsulation
   - Inheritance

Creating a Class

Example:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Creating an Object

Example:

```
p1 = Person("John", 30)
print(p1.name)  # Output: John
print(p1.age)   # Output: 30
```

Instance Attributes

- Defined inside methods like __init__.
- Unique to each object.

Example:

```
p2 = Person("Jane", 25)
print(p2.name)  # Output: Jane
```

Encapsulation

- Bundling data and methods within a class.
- Hiding implementation details.

Example: Encapsulation

Example:

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

Inheritance

- A class can inherit attributes and methods from another class.
- Promotes code reuse.

Example: Inheritance

Example:

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand)
        self.model = model
```

Polymorphism

- Objects can take on more than one form.
- A method behaves differently based on the object.

Example: Polymorphism

Example:

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def area(self):
        return "pi*r*r"

class Square(Shape):
    def area(self):
        return "a*a"
```

Abstraction

- Hiding complex implementation details.
- Using abstract classes or interfaces.

Example: Abstraction

Example:

```
# Abstract Base Classes
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        return "Bark"
```

Static Methods

Example:

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b

print(Math.add(5, 3))
# Output: 8
```

Class Methods

Example:

```
class Employee:
    company = "XYZ Corp"

    @classmethod
    def get_company(cls):
        return cls.company
```

Dunder Methods

- Special methods with double underscores, e.g., __str__.
- Provide custom behavior for built-in functions.

Example: Dunder Methods

Example:

```
class Book:
    def __init__(self, title):
        self.title = title

    def __str__(self):
        return f"Book: {self.title}"
```

FCPL - 11 - Object-Oriented Programming Languages 2

1. Object-oriented programming in C#
   - Introduction
   - Example
2. Object-oriented programming in Python
   - Python Classes and Objects
   - OOP Principles
   - Advanced OOP Features
3. Object-oriented programming in JavaScript
   - Objects in JavaScript
   - Classes and Objects
   - Encapsulation
   - Inheritance

Creating Objects

Example:

```
let person = {
    name: "John",
    age: 30,
    greet: function() {
        console.log("Hello " + this.name);
    }
};
person.greet(); // Output: Hello John
```

Accessing Properties

Example:

```
console.log(person.name); // Output: John
console.log(person['age']); // Output: 30
```

Defining a Class

Example:

```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
    greet() {
        console.log(‘Hello, my name is ${thi
    }
}
```

Creating an Object

Example:

```
let person = new Person("Alice", 25);
person.greet(); // Output: Hello, my name is
```

Adding Methods to Classes

Example:

```
class Car {
    start() {
        console.log("Car started");
    }
}
let car = new Car();
car.start(); // Output: Car started
```

Encapsulation in JavaScript

- Encapsulation bundles data and methods into objects.
- Access modifiers (e.g., private fields) control access.
- Helps hide implementation details.

Private Fields

Example:

```
class BankAccount {
    #balance = 0;

    deposit(amount) {
        this.#balance += amount;
    }

    getBalance() {
        return this.#balance;
    }
}
```

Implementing Inheritance

Example:

```
class Animal {
    speak() {
        console.log("Animal speaks");
    }
}

class Dog extends Animal {
    speak() {
        console.log("Dog barks");
    }
}
```

Understanding Polymorphism

- Polymorphism allows methods to behave differently based on the object calling them.
- Enables the same interface for different object types.

Polymorphism Example

Example:

```
class Shape {
    draw() {
        console.log("Drawing shape");
    }
}

class Circle extends Shape {
    draw() {
        console.log("Drawing circle");
    }
}
```

What is Abstraction?

- Abstraction hides complex implementation details.
- Provides a simple interface for the user.
- Achieved using abstract classes or interfaces.

Abstract Classes in JavaScript

Example:

```
class AbstractAnimal {
    makeSound() {
        throw new Error("Method 'makeSound(
    }
}

class Cat extends AbstractAnimal {
    makeSound() {
        console.log("Meow");
    }
}
```

Static Methods

Example:

```
class MathUtils {
    static add(a, b) {
        return a + b;
    }
}
console.log(MathUtils.add(2, 3)); // Output
```

Getters and Setters

Example:

```
class Rectangle {
    constructor(width, height) {
        this.width = width;
        this.height = height;
    }
    get area() {
        return this.width * this.height;
    }
}
let rect = new Rectangle(5, 10);
console.log(rect.area); // Output: 50
```

FCPL - 11 - Object-Oriented Programming Languages 2

1. Object-oriented programming in C#
   - Introduction
   - Example
2. Object-oriented programming in Python
   - Python Classes and Objects
   - OOP Principles
   - Advanced OOP Features
3. Object-oriented programming in JavaScript
   - Objects in JavaScript
   - Classes and Objects
   - Encapsulation
   - Inheritance

Bibliography

1. Horia Ciocarlie – The programming language universe, second edition, Timisoara, 2013.
2. Carlo Ghezzi, Mehdi Jarayeri – Programming Languages, John Wiley, 1987.
3. Ellis Horrowitz – Fundamentals of programming languages, Computer Science Press, 1984.
4. Donald Knuth – The art of computer programming, 2002.